

DEBOUNCE - Eliminación de rebotes de la señal emitida por un pulsador

Los pulsadores a menudo generan falsas señales de transición apertura/cierre cuando son pulsados debido a problemas mecánicos o físicos. Estas transiciones pueden ser leídas como si estuviéramos presionando el botón varias veces en un tiempo muy breve con lo cual podríamos alterar los resultados esperados en el programa.

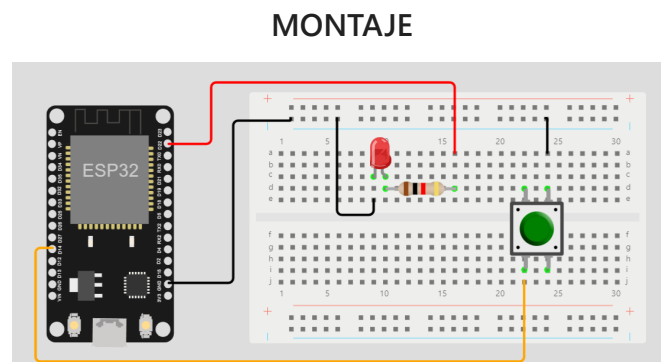


Se pueden evitar los rebotes por hardware, por software o combinando ambos sistemas, algunas soluciones pueden ser:

- Para evitarlos hardware se utilizan dispositivos electrónicos para filtrar señal. Lo más sencillo es colocar un condensador del orden de 1 μF en paralelo con el pulsador, que permitirá filtrar la mayoría del "ruido".
- Para evitarlos software se deberán ignorar las «pulsaciones» en el intervalo en que se produce el «ruido». Un intervalo de tiempo (threshold) de entre 100 y 200 ms suele ser adecuado, pero hay casos en los que se debe ajustar de forma que se elimine el rebote, pero no se ignoren posibles eventos cercanos «verdaderos»

Simulación

PASO 1: MONTAJE



PASO 2: Defino parámetros

```

from machine import Pin
import time

led_interno_azul=Pin(2, Pin.OUT)
led_externo_rojo=Pin(22, Pin.OUT)
pulsador=Pin(14, Pin.IN, pull=Pin.PULL_UP)

estado_led=1
estado_boton=0
ultimo_estado_de_boton=0
ultimo_tiempo_de_rebote=0
delay_por_rebote=50

```

PASO 3: Leo estado del pulsador

Ahora podemos dentro del bucle leer el estado del botón y almacenar el estado en una variable entera **lectura** y ver su valor en la consola:

```

while True:
    led_externo_rojo.value(estado_led)
    lectura=pulsador.value()

```

PASO 4: Recupero el tiempo en el cual cambia el estado del botón

```

if lectura!=ultimo_estado_de_boton:
    ultimo_tiempo_de_rebote=time.ticks_ms()
    print(ultimo_tiempo_de_rebote)

```

Al salir del condicional debo actualizar el estado del botón para poder registrar el próximo cambio de estado:

```

ultimo_estado_de_boton=lectura

```

PASO 5: Anulo tiempo de rebote

Si el tiempo transcurrido es mayor al tiempo en el cual se ha producido el cambio en el estado del botón en un valor prefijado considerado para eliminar el timepo de rebote prosigo:

```

if time.ticks_ms()-ultimo_tiempo_de_rebote>delay_por_rebote:

```

PASO 6: Prendo el led al presionar el botón

```

if lectura!=estado_boton:
    estado_boton=lectura
    if estado_boton==1:
        if estado_led==1:
            estado_led=0
        elif estado_led==0:
            estado_led=1

```

Código completo

```

from machine import Pin
import time

# definiciones

```

```
led_externo_rojo=Pin(22, Pin.OUT)
pulsador=Pin(14, Pin.IN, pull=Pin.PULL_UP)

estado_led=1
estado_boton=0 #POR AHORA NO USADA
ultimo_estado_de_boton=0
ultimo_tiempo_de_rebote=0
delay_por_rebote=50

while True:
    lectura=pulsador.value()
    if lectura!=ultimo_estado_de_boton:
        ultimo_tiempo_de_rebote=time.ticks_ms()
        print(">>>>>>",ultimo_tiempo_de_rebote)
    if time.ticks_ms()-ultimo_tiempo_de_rebote>delay_por_rebote:
        if lectura!=estado_boton:
            estado_boton=lectura
            if estado_boton==1:
                if estado_led==1:
                    estado_led=0
                elif estado_led==0:
                    estado_led=1

    led_externo_rojo.value(estado_led)
    ultimo_estado_de_boton=lectura
```